

第2章 Python序列

学习主要内容：

一、列表

- 1.切片操作
- 2.列表排序， 逆序
- 3.用于序列操作的常用内置函数
- 4.列表推导式

2.1 列表

列表方法

list slicing

方法	说明
<code>list.append(x)</code>	将元素x添加至列表尾部
<code>list.extend(L)</code>	将列表L中所有元素添加至列表尾部
<code>list.insert(index, x)</code>	在列表指定位置index处添加元素x
<code>list.remove(x)</code>	在列表中删除首次出现的指定元素
<code>list.pop([index])</code>	删除并返回列表对象指定位置的元素
<code>list.clear()</code>	删除列表中所有元素，但保留列表对象，该方法在Python2中没有
<code>list.index(x)</code>	返回值为x的首个元素的下标
<code>list.count(x)</code>	返回指定元素x在列表中的出现次数
<code>list.reverse()</code>	对列表元素进行原地逆序
<code>list.sort()</code>	对列表元素进行原地排序
<code>list.copy()</code>	返回列表对象的浅拷贝，该方法在Python2中没有

```
list_var[index]  
x in list_var  
x not in  
list_var
```

列表数据的切片

切片

- 切片是Python序列的重要操作之一，适用于列表、元组、字符串、range对象等类型。

slice[start:stop:step]

切片

- **功能**：可以使用切片来**截取**列表中的任何部分，得到一个**新列表**【切片返回的是列表元素的浅拷贝】，也可以通过切片来**修改**和**删除**列表中部分元素，甚至可以通过切片操作为列表对象**增加**元素。

切片

- **参数**：切片使用2个冒号分隔的3个数字来完成，第一个数字表示切片开始位置，第二个数字表示切片截止（但不包含）位置，第三个数字表示切片的步长，当步长省略时可以顺便省略最后一个冒号如：`aList[start : stop]`。

aList[start : stop : step]

回忆：

index在 $[0, \text{len}(s)-1]$

step可缺省为1，则表示列表中从start开始，每隔step的元素，直到stop（不包括）为止。

$[-\text{len}(s), -1]$

- 当step为正数，即 $\text{step} > 0$ ，表示**从左往右**，此时start缺省为0（第一个元素），stop缺省为 $\text{len}(s)$

如若start、stop表示的元素的下标转成同号，

- 若 $\text{start} < \text{stop}$ （start表示的元素在stop的左边），则切片为顺序；
- 若 $\text{start} \geq \text{stop}$ ，则切片为空，如 $a[-1:-4]$ 、 $a[4:4]$ 、 $a[4:1]$ 均返回空表。

- 当step为负数，即 $\text{step} < 0$ ，表示**从右往左**，此时start缺省为-1（最后一个元素），而stop缺省为列表开始，即 $-\text{len}(s)-1$

如若start、stop表示的元素的下标转成同号，

- 若 $\text{start} \leq \text{stop}$ （start表示的元素在stop的左边），则切片为空；如： $a[1:4:-1]$ 返回空表
- 若 $\text{start} > \text{stop}$ ，则切片为倒序。如 $s=[1,2,3,4]$ $s[-1:-4:-1] \rightarrow [4, 3, 2]$

- 注：**当start、stop符号不同且未超出索引范围时，可以通过“负数值+ $\text{len}(\text{aList})$ ”的方法，使得负数变成正数。

$a[1:-3]$ 与 $a[1:-3+\text{len}(a)]$ 一样，

$b[-4:-2]$ 与 $b[-4+\text{len}(b):-2+\text{len}(b)]$ 一样

aList[start : stop : step]

红点表示下标的有效取值 (start, step同号)

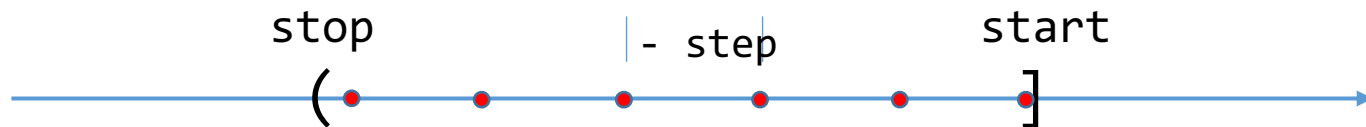
1. $\text{start} < \text{stop}, \text{step} > 0$



2. $\text{start} \geq \text{stop}, \text{step} > 0$ (列表为空)



3. $\text{start} > \text{stop}, \text{step} < 0$



4. $\text{start} \leq \text{stop}, \text{step} < 0$ (列表为空)



课堂练习：切片参数

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(a[2:3])
```

```
print(a[5:9])
```

```
print(a[5:-1])
```

```
print(a[-5:9])
```

```
print(a[-5:-1])
```

切片允许start,stop越界,仅返回能遍历到的元素

`s=[1,2,3,4]`

`s[-100:100]` #返回 `[1,2,3,4]` ,等价`s[0:4]`,

`s[-100:-200]` #返回 `[]` ,等价于`s[0:0]`

`s[100:200]` #返回 `[]` ,等价于`s[4:4]`

`s[:100]` #返回 `[1,2,3,4]` , 等价于`s[0:4]`

`s[100:-100:-1]` #返回 `[4,3,2,1]` 等价于`s[-1:-5:-1]`

课堂练习：切片范围

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(a[-100:5])
```

```
print(a[5:100])
```

```
print(a[-100:100])
```

```
print(a[100:1000])
```

2.1.6 切片操作

- **功能1**：可以使用切片来截取列表中的任何部分，得到一个新列表。

`s[start:stop:step]`: `step`缺省为1，则表示列表中从`start`开始，每隔`step`的元素，直到`stop`（不包括）为止，仅返回能遍历到的元素。

1. `step > 0`， **从左往右**
2. `step < 0`， **从右往左**

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[::2] #aList[0:10:2]一般写成 newList = aList[::2]
[3, 5, 7, 11, 15]
>>> aList[1::2] #aList[1:10:2]
[4, 6, 9, 13, 17]
>>> aList[0:100:1]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[100:] #aList[100:10:1]
[]
>>> aList[1:-1]
[4, 5, 6, 7, 9, 11, 13, 15]
```

```
>>> aList[3::] #aList[3:10:1]
[6, 7, 9, 11, 13, 15, 17]
>>> aList[3:6:-1]
[]
>>> aList[-1:6:-2] # aList[10:6:-2]
[17, 13]
```

```
>>> aList[::] # 等同于 aList[0:10:1]
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList[::-1] # 不等于 aList[0:10:-1] 等同于aList [-1:-11:-1]
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

2.1.6 切片操作

- 切片返回的是列表元素的浅拷贝

`bList = aList[:]`也可

```
>>> aList = [3, 5, 7]
```

```
>>> bList = aList[:] #浅复制
```

```
>>> aList == bList
```

```
True
```

```
>>> aList is bList
```

```
False
```

```
>>> id(aList) == id(bList)
```

```
False
```

```
>>> bList[1] = 8
```

```
>>> bList
```

```
[3, 8, 7]
```

```
>>> aList
```

```
[3, 5, 7]
```

- 浅拷贝

仅仅copy了原有列表各个元素（即对各个元素的引用），即新列表和原列表相同位置的元素都指向同一个对象。

与前面`list()`、`list.copy()`的描述类似

```
list_a = [1,2,[4,5]]
```

```
list_b = list_a[:]
```

```
list_a[2][0] = 5
```

```
print(list_b) #output [1,2,[5,5]]
```

2.1.6 切片操作

- **功能2：可以使用切片来原地修改列表内容：切片下标对应的元素如果不是空，替换或者删除元素**

- ✓ 给列表的切片下标赋值`list_var[slice]`时相当于将下标指定位置的元素替换为赋值语句右边的元素
- ✓ 右边为空列表时相当于删除对应的元素

*步长不为1时要求左右两边的元素个数一样

```
>>> aList = [3, 5, 7, 9, 10]
>>> aList[:3] = [1, 2, 3,4]    # 前面3个
元素移走后替换，后面的元素相应移位。
>>> aList
[1, 2, 3, 4, 9, 10]
>>> aList[:3] = [] # 相当于删除前面3个元素
>>> aList
[4, 9, 10]
```

```
>>> aList = list(range(10))
>>> aList
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> aList[::2] =
[0]*(len(aList)//2) # 步长不为1时
要求左右两边的元素个数一样
>>> aList
[0, 1, 0, 3, 0, 5, 0, 7, 0, 9]
```

2.1.6 切片操作

- 功能2：可以使用切片来原地修改列表内容：切片下标对应的元素如果为空，添加新元素

```
>>> aList = [3, 5, 7]
>>> aList[len(aList):]
[]
>>> aList[len(aList):] = [9,10]    # 相当于extend([9,10])
>>> aList
[3, 5, 7, 9,10]
>>> aList[:0] = [-1,1]
# 在前面添加元素
>>> aList
[-1, 1, 3, 5, 7, 9, 10]
>>> aList[3:3] = [500]              #
# 第3个位置添加元素
>>> aList
```

- 对列表的切片下标赋值
list_var[slice]时相当于在指定位置添加新元素，步长必须为1
- 可以在尾部添加多个元素
- 可以在前面添加多个元素
- 可以在中间添加多个元素(start所指位置)

```
>>> aList = [3, 5, 7, 500, 600]
>>> aList[4:3]=[10000,20000]
>>> aList
[3, 5, 7, 500, 10000, 20000, 600]
```

2.1.6 切片操作

- **功能3：使用del与切片结合起来删除列表元素(特别是间隔删除)**

```
>>> aList = [3,5,7,9,11]
>>> del aList[:3] # 相当于 aList[:3] = []
>>> aList
[9, 11]
```

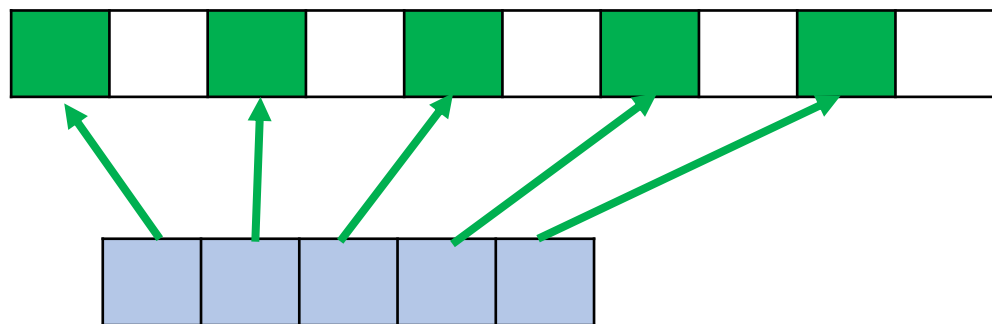
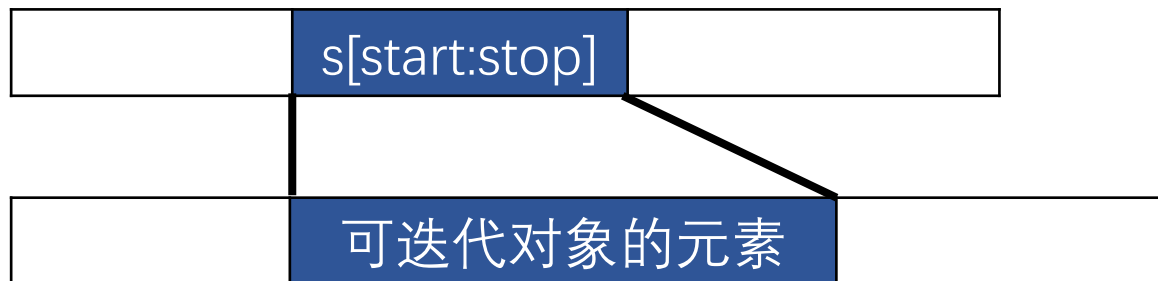
采用赋值语句来给切片赋值（包括右边为空列表来删除元素时）时，步长不为1时要求左右两边的元素个数一样。

```
>>> list1 = list(range(10))
>>> list1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list1[::2] = [1]* (len(list1)//2)
>>> list1
[1, 1, 1, 3, 1, 5, 1, 7, 1, 9]
>>> list1[::2] = [ ]
Traceback (most recent call last):
  File "<pysHELL#68>", line 1, in
<module>
    list1[::2] = [ ]
ValueError: attempt to assign sequence
of size 0 to extended slice of size 5
>>> del list1[::2]
>>> list1
[1, 3, 5, 7, 9]
```

总结:切片：原地修改列表

- 切片出现在表达式处，表示根据指定位置的元素构造一个新的列表
- 切片出现在**赋值语句左边**，表示原地修改(插入、替换或删除)列表中的元素

在下标i处插入多个元素：`list_var[i:i] = iterable`



步长不为1时要求左右两边的元素个数一样
→ 无法通过赋值来删除不连续的多个元素

替换或者删除元素

`s[start:stop] =
iterable`

`s[::2] =
iterable`

删除列表中的多个元素

`del s[start:stop]`
`del s[start:stop:step]`

课堂练习：切片

列表数据的操作

Sort

(1) 使用列表对象的`sort`方法进行原地排序，支持多种不同的排序方法。

```
help([].sort)
```

```
help(list.sort)
```

`List.sort(key=None, reverse=False) -> None`，默认基于元素间的大小关系，升序排列

2.1.7 列表排序

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> import random
```

```
>>> random.shuffle(aList)
```

```
>>> aList
```

```
[3, 4, 15, 11, 9, 17, 13, 6, 7, 5]
```

```
>>> aList.sort()
```

#默认为升序排列

```
>>> aList
```

```
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> aList.sort(reverse = True)
```

#降序排列

```
>>> aList
```

```
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

```
>>> def funcSort(x):
```

```
    return len(str(x))
```

```
>>> aList.sort(key=funcSort) #自定义排序
```

```
>>> aList
```

```
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
```

基于其对应的字符串长度升序排列，但是长度一致时并没有定义，当前实现按照现有顺序

2.1.7 列表排序

- (2) 使用**内置函数sorted**对列表进行排序并返回**新列表**
sorted(iterable, key=None, reverse=False)

```
>>> aList
```

```
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
```

```
>>> sorted(aList)
```

```
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> sorted(aList,reverse = True) #返回了新列表
```

```
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

```
>>> aList #列表本身没有发生变化
```

```
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
```

2.1.7 列表排序

(3) 使用列表对象的**reverse方法**将元素原地逆序

L.reverse() -- reverse *IN PLACE*

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> aList.reverse()
```

```
>>> aList
```

```
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

2.1.7 列表排序

(4) 使用**内置函数reversed方法**对列表元素进行逆序排列并返回新的**迭代器对象**

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> newList = reversed(aList)
```

```
>>> newList
```

```
<list_reverseiterator object at 0x0000000003624198>
```

```
>>> list(newList)      # 调用迭代器获得了所有内容
```

```
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

```
>>> for i in newList:
```

```
    print(i, end=' '),
```

无输出内容，迭代器已遍历结束，需要重新创建迭代器

```
>>> newList = reversed(aList)
```

```
>>> for i in newList:
```

```
    print(i, end=' ')
```

```
17 15 13 11 9 7 6 5 4 3
```

```
>>> list(newList) #迭代器已遍历结束，只能迭代一次 所以为空列表
```

```
[]
```



```
>>> newList = reversed(aList)
```

```
>>> next(newList)
```

```
17
```

```
>>> next(newList)
```

```
15
```

```
>>> next(newList)
```

```
13
```

```
>>> next(newList)
```

```
11
```

```
>>> next(newList)
```

```
9
```

```
>>> next(newList)
```

```
7
```

```
>>> next(newList)
```

```
6
```

```
>>> next(newList)
```

```
5
```

```
>>> next(newList)
```

```
4
```

```
>>> next(newList)
```

```
3
```

```
>>> next(newList)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#51>", line 1, in <module>
```

```
    next(newList)
```

```
StopIteration
```

```
>>>
```

next(newList)返回
迭代器下一个值，
最后抛出
StopIteration

课堂练习：数据排序

```
help(list.reverse)
```

```
a_list = [123, 'xyz', 'zara', 'abc', 'xyz']
```

```
a_list.reverse()
```

课堂练习：数据排序

```
aList = ['123', 'Google', 'Runoob', 'Taobao',  
         'Facebook']  
aList.sort()  
print(aList)  
aList.sort(reverse=True)  
print(aList)
```

课堂练习：数据排序

获取列表的第二个元素

```
def takeSecond(elem):  
    return elem[1]
```

列表

```
random = [(2, 2), (3, 4), (4, 1), (1, 3)]
```

指定第二个元素排序

```
random.sort(key=takeSecond)
```

输出类别

```
print(random)
```

2.1.8 用于序列操作的常用内置函数

- 序列大小比较：可以直接使用关系运算符来比较数值或序列的大小，也可以使用对象的“__le__()”及其相关方法（不建议使用）。
 - 相同类型的对象才能比较，数字类型（bool、整数、浮点等）的对象之间可以比较
 - 有序序列的对象可以比较，按顺序比较各个元素的大小，如果前面比较确定了大小关系，则结束。如果其中某个对象没有对应的元素，则无元素表示其最小
 - 字符之间的比较按照Unicode顺序比较，即ord(char)之间的比较。 空格... 0...9 ...A...Z... a...z ...

```
>>> a = [1, 2]
>>> b = [1, 2, 3]
>>> a.__le__(b)
True
>>> a.__gt__(b)
False
>>> a > b
False
>>> a < b
True
```

```
>>> [1, 2, 3] < [1, 2, 4]      #[1, 2,
3).__le__([1, 2, 4])
True
>>> 'ABC' < 'C' < 'Pascal' < 'Python'
True
>>> (1, 2, 3, 4) < (1, 2, 4)
True
>>> (1, 2) < (1, 2, -1)
True
>>> (1, 2, 3) == (1.0, 2.0, 3.0)
True
>>> (1, 2, ('aa', 'ab')) < (1, 2, ('ab', 'a'), 4)
True
>>> ord("A")
65
>>> ord("a")
97
>>>
```

复旦大学版权所有

2.1.8 用于序列操作的常用内置函数

- `len(列表)`：返回列表中的元素个数，同样适用于元组、字典、字符串等等。
- `max(列表)`、`min(列表)`：返回列表中的最大或最小元素，同样适用于元组、`range`、字符串等。
- `sum(列表)`：对数值型列表的元素进行求和运算，**对非数值型列表运算则出错**，同样适用于元组、`range`。

补充：元组(Tuple)

- 元组和列表类似，但属于不可变序列。
- 元组一旦创建，用任何方法都不可以修改其元素（增加、删除、赋值）。
- 元组的定义方式和列表相同，但定义时所有元素是放在一对圆括号 “(” 和 “)” 中，而不是方括号中。
- 在不引起歧义时，括号可省略

```
>>>a_tuple = ('a', )
>>> a_tuple
('a')
>>>a_tuple = ('a', 'b', 'mpilgrim',
'z', 'example')
>>> a_tuple
('a', 'b', 'mpilgrim', 'z', 'example')
>>> x = ()          #空元组
>>> x
()
>>> x = 1,2,(3,4)
>>> x
(1, 2, (3, 4))
```

注意：

如果创建只有一个元素的元组，需要在元素后面加上一个逗号 “,” 。

```
>>> a=(3)          ←-   解释为表达式
>>> a
3
>>> a=3,
>>> a
(3,)
```

2.1.8 用于序列操作的常用内置函数

- **zip(列表1,列表2,...)**:将多个列表对应位置元素组合为元组，并返回包含这些元组的zip对象（**迭代器**）。Zip对象可以进一步用list()函数转换为列表对象。

```
>>> aList = [1, 2, 3, 8]
```

```
>>> bList = [4, 5, 6]
```

```
>>> cZip = zip(aList, bList)
```

```
>>> cZip
```

```
<zip object at 0x0000000003728908>
```

```
>>> list(cZip)
```

```
[(1, 4), (2, 5), (3, 6)]
```

```
>>> list(cZip) #迭代器已遍历结束，只能迭代一次.所以为空列表
```

```
[]
```

2.1.8 用于序列操作的常用内置函数

- **enumerate(列表, start = 0)**：枚举列表元素，返回枚举对象（**迭代器**），其每个元素为包含下标和值的元组。该函数对元组、字符串同样有效。

```
>>> dList = [5, 6, 7]
>>> eEnum = enumerate(dList, 1)
>>> eEnum
<enumerate object at 0x00000218EABB3B80>
>>> for item in eEnum:
    print(item)
(1, 5)
(2, 6)
(3, 7)
>>> list(eEnum) # 迭代器已遍历结束，只能迭代一次. 所以为空列表
[]
```

补充：可迭代对象、迭代器

- iterable对象包括：**str,list,tuple,dict,set,range**等
- iterator包括：**reversed(),zip(),enumerate()** 等返回的对象
- 内置函数 `iter(iterable)`可以返回一个iterator (不需要掌握)
- 内置函数`next(iterator)`从迭代器返回下一个元素，没有更多的元素时抛出异常`StopIteration`
- 二者都可以用for 循环进行遍历各个元素
 - **for var in iterator(or iterable)** 循环相当于每次取可迭代对象或者迭代器的下一个元素，执行一系列语句，然后取下一个元素执行，直到最后`StopIteration`时结束

列表推导式

2.1.9 列表推导式

- 集合论： $\{x \mid p(x)\}$
 - 偶数 $\{x \mid x = 2k, k \in \mathbb{Z}\}$
 - 平方数 $\{x \mid x = k^2, k \in \mathbb{Z}\}$ 或者 $\{x^2 \mid x \in \mathbb{Z}\}$
- 列表推导式/解析式 (list comprehension) 是利用其他列表[创建新列表](#)的一种方法，非常简洁，代码具有强可读性

语法： `[表达式 for 变量 in 序列]` 或者 `[表达式 for 变量 in 序列 if 条件]`
`[ 2*k for k in range(100)]` `[ 2*k for k in range(100) if k % 10 == 0]`

- 过程式编程(procedural programming)：问题分解成多个步骤，给出每个步骤执行的指令
- 函数式编程(functional programming)：把问题分解成多个子问题（函数），对于每个函数，给定输入，产生希望的输出，不用关心函数内部怎么计算 → 列表推导式和生成式是函数式编程的例子

2.1.9 列表推导式：一个for子句

- 列表推导式：对于序列中的每个成员（通过if条件筛选）计算一个新的表达式后保存在一个新列表里
- 语法：**[表达式 for 变量 in 序列]** 或者 **[表达式 for 变量 in 序列 if 条件]**

```
aList = [x*x for x in range(10)]  
>>> aList  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

相当于

```
aList = []  
for x in range(10):  
    aList.append(x*x)
```

```
list1 = [ 2*k for k in range(100) if k % 10 == 0]  
>>> list1  
[0, 20, 40, 60, 80, 100, 120, 140, 160, 180]
```

```
def list_comprehension_imp(expr_func, seq, cond):  
    new_list = []  
    for item in seq:  
        if cond(item):  
            new_list.append(expr_func(item))  
    return new_list  
list2 = list_comprehension_imp(lambda x: 2*x,  
                                range(100),  
                                lambda x: x % 10 == 0)
```

不作要求!!!

复旦大学版权所有

2.1.9 列表推导式：例子

- 立方数：

```
>>> [x**3 for x in range(1,20,2)]
```

```
[1, 27, 125, 343, 729, 1331, 2197, 3375, 4913, 6859]
```

- 过滤不符合条件的元素：仅仅包含正数

```
>>> aList = [-1,-4,6,7.5,-2.3,9,-11]
```

```
>>> [i for i in aList if i>0]
```

```
[6, 7.5, 9]
```

- 列出当前文件夹下所有Python源文件

```
>>> [filename for filename in os.listdir( '.' ) if filename.endswith( '.py' )]
```

#现不作要求，文件学后

2.1.9 列表推导式：复杂表达式

列表推导式可以使用函数或复杂表达式

- 元素求绝对值

```
>>> vec = [-1,-4,6,7.5,-2.3,9,-11]
```

```
>>> [abs(x) for x in vec]
```

```
[1, 4, 6, 7.5, 2.3, 9, 11]
```

- 去掉前后的空格类字符现不作要求（字符串学习后）

```
>>> freshfruit = [' banana', ' loganberry ', 'passion fruit ']
```

```
>>> [weapon.strip() for weapon in freshfruit]
```

```
['banana', 'loganberry', 'passion fruit']
```

- 每个元素为元组，(数，数的平方)

```
>>> [(x, x**2) for x in range(6)]
```

```
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

- if else表达式

```
>>> print([v**2 if v%2 == 0 else v+1 for v in [2, 3, 5, 4, -1] if v>0])#现不作要求选择分支学后
```

```
[4, 4, 6, 16]
```

2.1.9 列表推导式：复杂表达式(续)

返回一个名字，长度介于min_chars和max_chars之间

```
import random
import string
def get_name(min_chars = 3, max_chars = 8):#现不做要求，字符串学后
    VALID_CHARS = string.ascii_lowercase + 'abcdefghijklmnopqrstuvwxyz'
    names = [random.choice(VALID_CHARS) for i in range(random.randint(min_chars, max_chars))]
    names[0] = names[0].upper()
    return ''.join(names)
```

```
>>> get_name()
'Ggypmte'
>>> get_name()
'Wymxqfhg'
>>> get_name(3, 8)
'Pbnbdr'
>>> get_name(3, 8)
'Quxzjshx'
>>>
```

产生一个班级（人数在45-90之间）的分数

```
exam_scores = [random.randint(50,100) for i in range(random.randint(45,90))]
```

```
>>> exam_scores
[95, 53, 73, 76, 55, 82, 66, 79, 74, 92, 59, 79, 66, 50, 50, 53, 87, 93, 61, 60, 69,
88, 64, 67, 53, 52, 51, 76, 65, 71, 66, 66, 60, 66, 81, 56, 65, 98, 66, 76, 72, 55, 7
5, 89, 75, 88, 53, 55, 59, 88, 60, 86, 83, 57, 85, 81, 70, 71, 65, 90, 81, 68, 51, 93
, 75, 86, 75, 83, 88, 76]
>>> len(exam_scores)
73
>>>
```

2.1.9列表推导式:多个for/if子句

- 语法：[表达式 for 变量 in 列表] 或者 [表达式 for 变量 in 列表 if 条件]
- 第一个for之后可以跟0或者多个for或者if子句

```
[expr for value in seq1 if condition1  
      for value2 in seq2 if condition2  
      ...  
      for valueN in seqN if conditionN ]
```

生成两位数偶数的列表

```
list1 = [ x*10+y for x in range(10) if x != 0  
          for y in range(10) if y % 2 == 0]
```

```
>>> list1  
[10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48,  
 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88,  
 90, 92, 94, 96, 98]  
>>>
```

```
[expr for value in seq1 if condition1
    for value2 in seq2 if condition2
    ...
    for valueN in seqN if conditionN ]
```

```
def list_comprehension_multi_imp(expr_func1, seq1, cond1,
                                  seq2, cond2):
    new_list = [ ]
    for item in seq1:
        if cond1(item):
            for j in seq2:
                if cond2(item, j):
                    new_list.append(expr_func1(item, j))
    return new_list
```

2.1.9列表推导式:多个for/if子句

生成两位数偶数的列表

```
list1 = [ x*10+y for x in range(10) if x != 0
          for y in range(10) if y % 2 == 0]
```

```
>>> list1
[10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48,
 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88,
 90, 92, 94, 96, 98]
```

```
def list_comprehension_multi_imp(expr_func1, seq1, cond1,
                                  seq2, cond2):

    new_list = [ ]
    for item in seq1:
        if cond1(item):
            for j in seq2:
                if cond2(item, j):
                    new_list.append(expr_func1(item, j))
    return new_list
```

不作要求!!!

```
list2 = list_comprehension_multi_imp(lambda x,y: x*10+y,
                                       range(10), lambda x: x != 0,
                                       range(10), lambda x,y: y % 2 == 0)
```

2.1.9 列表推导式:多个for/if子句

应用举例：

- 在列表推导式中使用多个循环，实现多序列元素的任意组合，并且可以结合条件语句过滤特定元素

```
>>> [(x, y) for x in range(3) for y in range(3)]
```

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

```
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
```

```
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

#使用普通方法实现

```
combs = []  
for x in [1,2,3]:  
    for y in [3,1,4]:  
        if x != y:  
            combs.append((x, y))
```

2.1.9 列表推导式：多个for/if子句(续)

- 使用列表推导式实现嵌套列表的平铺，即新列表由最内层的列表元素组成

```
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- 使用列表推导式的嵌套来实现矩阵转置

```
>>> matrix = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
>>> [ row[i] for row in matrix for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

```
[1, 2,
 3, 4],
[5, 6,
 7, 8],
[9, 10, 11,
 12]
```

- 也可以使用内置函数来实现矩阵转置

```
>>> list(zip(*matrix)) #现不作要求，函数调用时的序列解包
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

```
list(zip(matrix[0], matrix[1], ..., matrix[1
```

2.1.9 列表推导式：练习1

- 使用列表推导式生成100以内的所有素数

```
>>> [ p for p in range(2, 100)
      if 0 not in [ p % d for d in range(2,int(sqrt(p))+1)] ]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
83, 89, 97]
```

提示1：必须导入math模块，使用sqrt函数

提示2：

```
[ p % d for d in range(2, int(sqrt(p))+1)]
```

p %d 得到的余数列表,如果0在其中说明p可以分解, 不为素数

列表推导式：练习2

毕达哥拉斯三元数组：存在 $\{x,y,z\}$, $0 < x < y < z$, 使得 $x^2 + y^2 = z^2$
求前nums个毕达哥拉斯三元数组

```
def pyt_firstN_list(nums):  
    pyt = [(x,y,z) for z in range(100)  
            for y in range(1,z)  
            for x in range(1,y) if x*x + y*y == z*z ]  
    firstN_pyt = pyt[:nums]  
    print(firstN_pyt)
```

```
pyt = [(x,y,z) for z in range(1000) for y in range(1,z) \  
        for x in range(1,y) if x*x + y*y == z*z ]
```

是不是很慢！如何解决呢？？

2.1.9 列表推导式: 练习3

dict里面讲解*

例如, 已知有一个包含一些同学成绩的字典, 计算成绩的最高分、最低分、平均分, 并查找所有最高分同学, 代码可以这样编写:

```
>>> scores = {"Zhang San": 45, "Li Si": 78, "Wang Wu": 40, "Zhou Liu": 96, "Zhao Qi": 65, "Sun Ba": 90, "Zheng Jiu": 78, "Wu Shi": 99, "Dong Shiyi": 60}
>>> highest = max(scores.values())
>>> lowest = min(scores.values())
>>> highest
99
>>> lowest
40
>>> average = sum(scores.values())/len(scores)
>>> average
72.33333333333333
>>> highestPerson = [name for name, score in scores.items() if score == highest]
>>> highestPerson
['Wu Shi']
```

课堂作业:

题目1：有五个数字：1,2,3,4,0，能组成多少个互不相同且无重复数字的三位数，各是多少？结果逐行打印于命令行。

题目2：使用列表推导式构建：[[1, 4, 9], [16, 25, 36], [49, 64, 81]]。

题目3：已知扑克牌的列表数据，请生成一副牌，并洗牌

```
ranks=['2','3','4','5','6','7','8','9','10','J','Q','K','A']
```

```
king=['大王','小王']
```

```
suits=["红桃","黑桃","方片","草花"]
```